UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

Sistemas Distribuidos

Clase 08 — De Kafka y causalidad a consenso, CAP, Dynamo y Spanner

Departamento de Computación
Ingeniería Informática – Comisión A – Pilar
Equipo docente – 2026

La historia que guiará toda la clase

NÚCLEO

En esta clase seguiremos el destino de una única entrada VIP que Sofía y Tomás intentan comprar casi al mismo tiempo desde regiones diferentes.

Lo que necesitamos comprender

- qué eventos están relacionados y cuáles son concurrentes;
- cómo varias réplicas acuerdan una decisión;
- qué ocurre cuando la red impide coordinar;
- qué costo paga cada garantía.

La pregunta transversal

¿Qué error es inaceptable y qué costo aceptamos para evitarlo?

latencia · indisponibilidad · coordinación · reparación

Evidencias de aprendizaje

NÚCLEO

Durante la clase

- construir *happened-before* y reconocer concurrencia;
- completar relojes de Lamport y vectoriales;
- seguir una entrada Raft y predecir un crash o una partición.

Al cerrar

- distinguir mayoría de consenso de quorum Dynamo-like;
- decidir cuándo un conflicto puede fusionarse;
- explicar TrueTime y defender una política con falla, costo y métrica.

La evidencia será reconstruir una ejecución y justificar por qué el sistema confirma, espera, rechaza, conserva versiones o repara.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Puente desde Kafka

Sistemas Distribuidos

Repaso visual: un Mundial en tiempo real

NÚCLEO



Idea recuperada

Kafka preserva una secuencia de registros para que productores y consumidores avancen de manera desacoplada y recuperable.

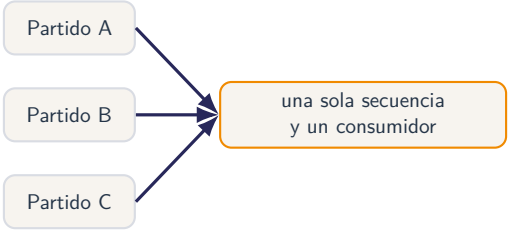
Una única secuencia no alcanza para muchos partidos

Problema de capacidad

Muchos partidos compiten por la CPU, red y almacenamiento de una única secuencia.

Problema de orden

Distribuir eventos al azar puede mostrar un gol antes de que comience su partido.



La key define el ámbito de orden

NÚCLEO

Decisión

Usamos `game_id` como key: todos los eventos de un partido permanecen en la misma partición.

P0	A: Started → YellowCard
P1	B: Started → Substitution
P2	C: Started → Goal

Ganamos orden local por partido y paralelismo entre partidos. No obtenemos un offset global gratuito entre P0, P1 y P2.

Consumer groups: repartir trabajo y releer la misma historia

NÚCLEO

Grupo `live-score`

- P0 → Score-1
- P1 → Score-2
- P2 → Score-3

Las particiones se reparten dentro del grupo.

Grupo `analytics`

Lee nuevamente P0, P1 y P2, pero conserva sus propios offsets.

Una partición tiene una sola instancia activa dentro de un grupo; grupos diferentes pueden leer la misma partición con progreso independiente.

Crash, replay e idempotencia

NÚCLEO



Límite

At-least-once permite que el registro reaparezca; no garantiza por sí solo que el efecto lógico ocurra una única vez.

Algunas métricas a considerar en Kafka

NÚCLEO

Definiciones

- **RF=3**: líder + dos seguidores configurados.
- **ISR**: réplicas suficientemente sincronizadas.
- **minISR=2**: hacen falta al menos dos réplicas en ISR.
- **acks=all**: todas las réplicas del ISR actual confirman el batch.

Si Réplica3 sale del ISR, pero Réplica1 y Réplica2 siguen sincronizadas, la escritura continúa. Si solo queda Réplica1, se rechaza: no se reconoce éxito con una sola copia sincronizada.

Guardrail docente	Referencia
mensaje habitual	< 1 MB
datos retenidos/broker	~ 1 TB
ingreso conservador/broker	< 10 MB/s

Son presupuestos docentes, no máximos universales. Deben validarse con hardware, tamaño de registros, replicación y pruebas de carga.

Puente: Kafka ordena registros; no elige por sí solo una decisión autoritativa

NÚCLEO

Kafka aporta

- orden dentro de cada partición;
- replay y progreso por grupo;
- replicación líder–seguidor;
- desacoplamiento productor–consumidor.

La nueva pregunta

Si dos regiones intentan reservar la única entrada VIP, ¿qué comando debe convertirse en la decisión autoritativa?

Pasamos de preservar una historia a acordar qué historia será válida.

Por qué estos conceptos se usan hoy

Sistema real 1: Kubernetes y etcd

CASO REAL

Qué es Kubernetes

Plataforma que administra aplicaciones en contenedores y mantiene un estado deseado del cluster.

Qué es etcd

Almacén clave–valor distribuido donde el control plane conserva objetos y configuración autoritativa.

etcd utiliza Raft para acordar un log; sin mayoría no puede confirmar nuevos cambios del control plane.

Sistema real 2: Amazon Dynamo

CASO REAL

Qué fue Dynamo

Almacén clave–valor altamente disponible diseñado por Amazon para servicios que debían continuar respondiendo durante fallas.

Decisión de diseño

Conservar versiones concurrentes, responder con quórums flexibles y reparar después, en vez de bloquear siempre por coordinación global.

La disponibilidad se paga con versiones, reconciliación, hints y trabajo de reparación.

Sistema real 3: Google Spanner

CASO REAL

Qué es Spanner

Base de datos distribuida globalmente que ofrece transacciones entre grupos de datos y un orden compatible con lo observado por los clientes.

Decisión de diseño

Combinar consenso localizado, coordinación transaccional y TrueTime, una abstracción que expone incertidumbre temporal.

Spanner paga coordinación y espera temporal para ofrecer garantías globales más fuertes.

Lo que un ingeniero decide realmente

NÚCLEO

Operación	Invariante	Error tolerable	Evidencia
última entrada	confirmaciones ≤ 1	rechazo temporal	doble venta, p99
carrito	no perder intención	ramas fusionables	conflictos, convergencia
ranking	atraso acotado	vista anterior	lag temporal
ledger (registros contables)	no crear dinero	espera o rechazo	anomalías, abortos

La pregunta no es «¿qué tecnología es mejor?», sino «¿qué error es inaceptable para esta operación?».

Mapa de la clase

NÚCLEO

1. **Observar**: fallas parciales y conocimiento local.
2. **Describir**: causalidad y relojes lógicos.
3. **Decidir**: máquinas de estado, Raft y Paxos.
4. **Elegir durante una partición**: CAP y PACELC.
5. **Conservar o evitar conflictos**: Dynamo y Spanner.

UNIVERSIDAD AUSTRAL

INGENIERÍA

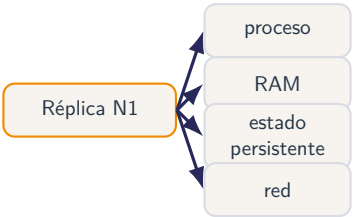
Departamento de Computación

Modelo mínimo y caso conductor

Sistemas Distribuidos

Modelo mínimo: qué contiene una réplica

NÚCLEO



Un nodo puede estar sano localmente y, aun así, ignorar qué ocurrió en otra región.

Tres situaciones que no debemos confundir

NÚCLEO

Sano y comunicado	Sano pero aislado	Caído
Ejecuta y recibe mensajes.	Ejecuta, pero sus mensajes no cruzan a tiempo.	No ejecuta pasos ni responde.

Desde otro nodo, un timeout puede ser compatible con cualquiera de estas historias.

Timeout: observación local, no diagnóstico

NÚCLEO



- la solicitud nunca llegó;
- B está lento o pausado;
- B ejecutó y la respuesta se perdió;
- la red está particionada.

El timeout solo prueba que A no obtuvo respuesta dentro de su plazo.

Caso conductor: Festival Austral Global

Recurso

`VIP-01`, stock $= 1$.

Clientes

Sofía compra en Buenos Aires; Tomás compra en Madrid.

Servicio replicado

Cinco réplicas: N1, N2, N3, N4 y N5.

Invariante

$$\text{reservas confirmadas para VIP-01} \leq 1$$

Predicción inicial

ACTIVIDAD

La red se divide:

$$\{N1, N2, N3\} \mid \{N4, N5\}$$

Sofía compra desde el lado de tres nodos y Tomás desde el lado de dos.

Elija antes de conocer el algoritmo

¿Qué debería responder el lado minoritario a Tomás?

A. Confirmada.

B. Rechazada temporalmente.

C. Pendiente hasta recuperar coordinación.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Causalidad y relojes lógicos

Sistemas Distribuidos

Happened-before: tres reglas

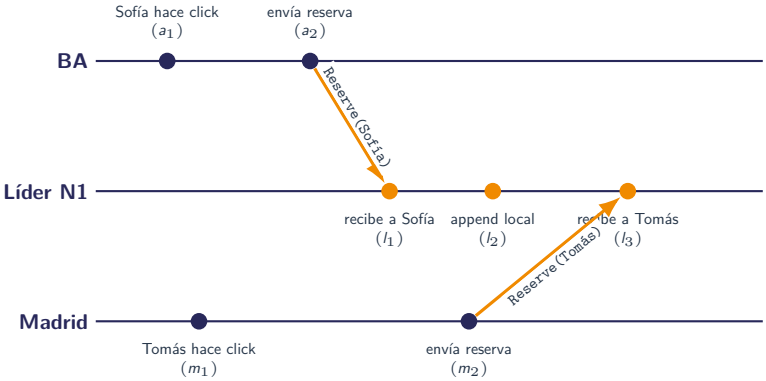
1. Si dos eventos ocurren en el mismo proceso, su orden local induce causalidad.
2. El envío de un mensaje ocurre antes que la recepción de ese mensaje.
3. La relación es transitiva.

$$a \to b \land b \to c \Rightarrow a \to c$$

significa que existe una cadena de influencia posible desde a hasta c.

Las dos reservas nacen en ramas independientes y llegan al líder

NÚCLEO



Lamport: convertir relaciones causales en números crecientes

Variables

L es el contador lógico local. t_m es la marca lógica incluida por el emisor en el mensaje.

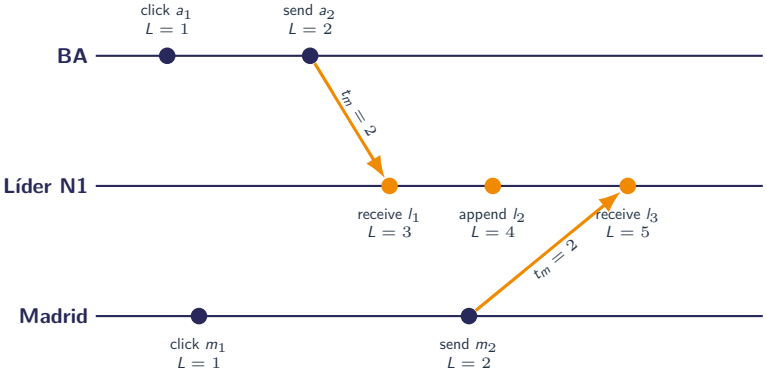
Al recibir:

$$L \leftarrow \max(L, t_m) + 1$$

- $\max$ conserva la historia local y la comunicada;
- $+1$ coloca la recepción estrictamente después del envío.

Ejemplo: $L = 4$, $t_m = 7 \Rightarrow$ nuevo $L = 8$.

La misma ejecución con relojes de Lamport



El líder comienza con $L = 0$:

Ejemplo Lamport sobre las dos reservas

ACTIVIDAD

Evento	Proceso	Lamport
Sofía presiona comprar	BA	1
BA envía reserva	BA	2
Tomás presiona comprar	Madrid	1
Madrid envía reserva	Madrid	2
Líder recibe a Sofía	líder	3
Líder hace append	líder	4
Líder recibe a Tomás	líder	5

Límite de Lamport

$$a \to b \Longrightarrow L(a) < L(b)$$

pero:

$$L(a) < L(b) \not\Longrightarrow a \to b.$$

Por qué

Los números de Lamport permiten respetar causalidad y construir un desempate total convencional, pero no distinguen por sí solos una dependencia real de una simple concurrencia.

Relojes vectoriales: registrar conocimiento por actor

NÚCLEO

Usamos el orden:

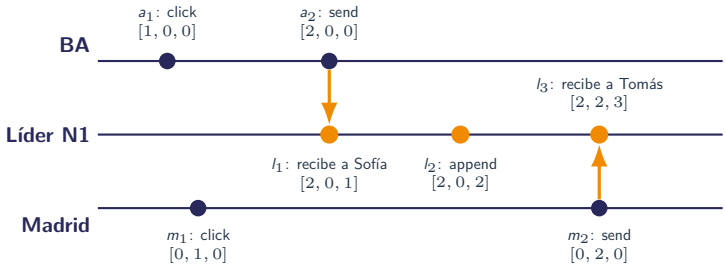
$$[\text{BA},\ \text{Madrid},\ \text{Líder}]$$

Cada proceso incrementa su componente y, al recibir, toma el máximo componente a componente antes de incrementar el suyo.

Comparación

$V_a < V_b$ si todos los componentes de V_a son menores o iguales y al menos uno es estrictamente menor. Si ninguno domina al otro, son concurrentes.

La misma ejecución con relojes vectoriales



Orden de componentes

[BA, Madrid, Líder]

Cada proceso incrementa solamente su propio componente.

$$[2, 0, 0] \parallel [0, 2, 0]$$

Los vectores de las dos reservas son incomparables: ninguna reserva contiene conocimiento causal de la otra.

Cómo actualiza su reloj vectorial el líder

NÚCLEO

Regla de recepción

Al recibir un mensaje, el proceso:

1. calcula el máximo componente a componente entre su vector y el recibido;
2. incrementa su propio componente para representar el nuevo evento local.

$$V(l_1) = \max\big([0,0,0],[2,0,0]\big) + [0,0,1] = [2,0,1],$$
$$V(l_2) = [2,0,1] + [0,0,1] = [2,0,2],$$
$$V(l_3) = \max\big([2,0,2],[0,2,0]\big) + [0,0,1] = [2,2,3].$$

¿Qué expresa $[2,2,3]$? El líder conoce:

- dos eventos de Buenos Aires;
- dos eventos de Madrid;
- tres eventos locales propios.

El resultado es $[2,2,3]$, no $[2,2,5]$.
El valor 5 pertenece al reloj escalar de Lamport.
En el reloj vectorial, el tercer componente cuenta solamente los eventos locales del líder.

Vector clocks en la historia

NÚCLEO

Evento	Vector
BA envía reserva de Sofía	$[2, 0, 0]$
Madrid envía reserva de Tomás	$[0, 2, 0]$
Líder recibe a Sofía	$[2, 0, 1]$
Líder hace append	$[2, 0, 2]$
Líder recibe a Tomás	$[2, 2, 3]$

El reloj escalar del líder termina en $L = 5$, pero el tercer componente vectorial es 3 porque cuenta sus tres eventos locales. No se mezclan ambos relojes.

Tres relojes, tres preguntas

NÚCLEO

Reloj	Pregunta	Uso
pared	¿qué fecha y hora civil es?	auditoría, timestamps humanos
monotónico	¿cuánto duró una espera?	timeouts, latencias, leases
lógico	¿qué orden causal debe respetarse?	causalidad, versiones, logs

Los relojes describen relaciones. Todavía no deciden quién obtiene VIP-01.

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

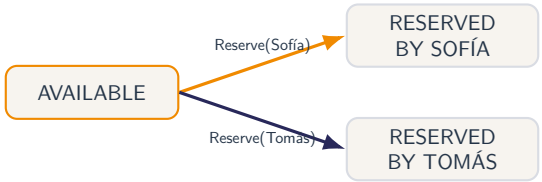
Máquinas de estado y consenso

Sistemas Distribuidos

Máquina de estados: una regla determinista

NÚCLEO

$$(\textit{estado}, \textit{comando}) \longrightarrow (\textit{nuevo estado}, \textit{respuesta})$$



Una reserva posterior sobre un estado ya reservado se rechaza sin cambiar el estado.

Máquina de estados replicada

Cada réplica contiene:

- una copia del log de comandos;
- el mismo código determinista;
- una copia local del estado.

Si todas aplican los mismos comandos comprometidos en el mismo orden, convergen al mismo estado.

Safety y liveness

NÚCLEO

Safety: nada incorrecto ocurre

- no hay dos comandos distintos comprometidos en el mismo índice;
- una entrada committed no se reemplaza;
- VIP-01 no se confirma para dos personas.

Liveness: el sistema vuelve a progresar

Requiere mayoría comunicada, líder estable, procesos activos y entrega eventual de los mensajes necesarios.

Raft: término, índice y entrada de log

- **Término**: número creciente que identifica una ronda de elección y liderazgo.
- **Índice**: posición estable de una entrada dentro del log.
- **Entrada**: par $(\mathit{term}, \mathit{command})$ en una posición.

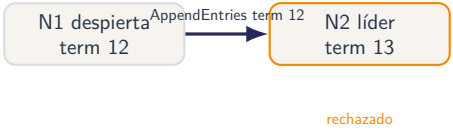
Ejemplo

Los índices 1–86 ya existen. El líder del término 12 agrega:

$$\mathit{index} = 87, \quad \mathit{term} = 12, \quad \mathit{command} = \mathrm{Reserve}(\text{VIP-01}, \text{Sofía})$$

Mensaje o líder obsoleto

Un mensaje es obsoleto cuando contiene un término menor que el término ya conocido por el receptor.



Raft paso 1: la propuesta existe solo en N1

NÚCLEO

Nodo	Última entrada	Estado
N1	87 / term 12 / Reserve(Sofía)	local
N2	hasta 86	—
N3	hasta 86	—
N4	hasta 86	—
N5	hasta 86	—

Todavía no

Una propuesta local no autoriza responder «compra confirmada».

Raft paso 2: dos copias todavía no son mayoría

NÚCLEO

Nodo	Última entrada	Estado
N1	87 / term 12 / Reserve(Sofía)	local
N2	87 / term 12 / Reserve(Sofía)	local
N3	hasta 86	—
N4	hasta 86	—
N5	hasta 86	—

N2 persiste la entrada y responde al líder.

$$2/5 < 3/5$$

La seguridad no proviene solo del número de copias, sino de una mayoría intersectante y de las reglas del protocolo.

Raft paso 3: la tercera réplica cambia la situación

Nodo	Última entrada	Estado
N1	87 / term 12 / Reserve(Sofía)	local
N2	87 / term 12 / Reserve(Sofía)	local
N3	87 / term 12 / Reserve(Sofía)	local
N4	hasta 86	—
N5	hasta 86	—

N3 persiste la misma entrada.

$$3/5$$

Toda mayoría futura de tres nodos debe compartir al menos un nodo con $\{N1, N2, N3\}$.

La entrada puede avanzar a committed porque alcanzó la mayoría exigida por el protocolo.

Raft paso 4: quién compromete, aplica y responde

NÚCLEO

1. N2 y N3 confirman al líder que persistieron la entrada.
2. N1 detecta mayoría y actualiza `commitIndex=87`.
3. N1 aplica la entrada en **su propia** máquina de estados.
4. N1 responde «compra confirmada» a Sofía.
5. N2 y N3 reciben `leaderCommit=87`, avanzan y aplican localmente.

Replicada $\neq$ committed $\neq$ aplicada $\neq$ confirmada al cliente.

Crash antes y después de la mayoría

ACTIVIDAD

Antes de mayoría

N1 contiene la entrada y cae. No estaba committed y puede desaparecer. Sofía no debió recibir éxito.

Después de mayoría

N1, N2 y N3 persistieron; N1 confirmó y cae. N2 o N3 conserva la decisión comprometida.

Total-order broadcast: por qué aparece aquí

Necesidad

Máquinas deterministas solo convergen si reciben los mismos comandos comprometidos y los aplican en el mismo orden.

Relación con consenso

Consenso elige un valor para una posición. Repetir consenso para 87, 88, 89, ... construye un log ordenado; ese servicio implementa total-order broadcast.

Un timestamp Lamport etiqueta eventos, pero no garantiza que todas las réplicas reciban el mismo conjunto de comandos ni que no existan huecos.

Pausa de proceso, GC y fencing

NÚCLEO

- **GC**: pausa del runtime para recuperar memoria; el proceso puede dejar de ejecutar sin estar muerto.
- **Fencing token**: número creciente que un recurso externo usa para rechazar operaciones de una autoridad antigua.

N1 líder term 12 $\rightarrow$ pausa $\rightarrow$ N2 líder term 13 $\rightarrow$ N1 despierta obsoleto

El término protege al cluster; un recurso externo puede requerir además un token de fencing.

Paxos: misma meta, organización diferente

EXTENSIÓN

Paso	Raft	Multi-Paxos
liderazgo	líder, term 12	proposer estable, ballot 12
posición	index 87	slot 87
propuesta	AppendEntries	Accept
réplicas	followers	acceptors
decisión	mayoría replica y commit	mayoría acepta, valor elegido
aplicación	máquina de estados	learner y máquina de estados

Ambos impiden decisiones incompatibles para la misma posición, aunque organizan roles y mensajes de forma diferente.

Caso real: dónde aparecen estas ideas en Kubernetes

CASO REAL

`kubectl` → API server → líder etcd → seguidores etcd

Concepto	Aplicación en etcd
líder y término	un líder Raft atiende escrituras y rechaza términos viejos
log e índice	cada cambio ocupa una posición del log replicado
mayoría y commit	se confirma tras alcanzar quorum
máquina de estados	todos obtienen el mismo key–value store
WAL/snapshot	cada miembro recuerda decisiones tras reiniciar

Tres logs que no debemos confundir

NÚCLEO

Log	Pregunta	Ejemplo
consenso	¿qué comando se acordó y en qué orden?	log Raft
WAL local	¿qué recuerda físicamente el nodo tras un crash?	WAL de la DB/etcd
eventos	¿qué hechos pueden leer y reprocesar otros componentes?	Kafka

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

CAP, PACELC y garantías

La partición ocurre después del commit de Sofía

$$\{N1, N2, N3\} \mid \{N4, N5\}$$

La mayoría conoce la reserva committed de Sofía. La minoría está sana localmente, pero no recibe ese hecho. Tomás intenta comprar desde Madrid.

Dos historias indistinguibles para la minoría

NÚCLEO

H1

Sofía ya fue confirmada, pero el mensaje no cruza la partición.

H2

Sofía nunca fue confirmada.

N4 y N5 observan lo mismo en ambas historias. Una respuesta local idéntica puede ser correcta en una e incorrecta en la otra.

CAP: la decisión durante una partición

NÚCLEO

- **Consistencia fuerte**: una sola historia compatible con una copia autoritativa.
- **Disponibilidad CAP**: cada solicitud a un nodo no fallado termina con una respuesta válida de la operación.
- **Partición**: grupos de nodos vivos no pueden intercambiar los mensajes necesarios a tiempo.

Durante la partición, confirmar localmente a Tomás arriesga doble venta; esperar o rechazar preserva el invariante, pero sacrifica disponibilidad CAP para esa operación.

Uptime, disponibilidad CAP y disponibilidad semántica

NÚCLEO

Concepto	Qué mide
uptime	fracción de una ventana mensual, trimestral o anual en que el proceso estuvo alcanzable
disponibilidad CAP	si la operación termina con respuesta válida durante una partición
disponibilidad semántica	fracción de intentos válidos que cumplen la promesa de negocio

Un 503 rápido demuestra que el proceso está vivo, pero no satisface disponibilidad CAP para la compra.

PACELC: el costo continúa cuando la red funciona

NÚCLEO

Lectura local

15 ms, pero puede observar una réplica atrasada.

Lectura coordinada

180 ms, con garantía más fuerte.

PACELC organiza dos preguntas: durante una partición, consistencia o disponibilidad; en régimen normal, latencia o consistencia.

Recencia, linearizabilidad y serializabilidad

NÚCLEO

Concepto	Definición breve
recencia	qué tan nuevo es el estado observado
linearizabilidad	cada operación parece atómica entre invocación y respuesta y respeta el orden real de operaciones no superpuestas
serializabilidad	el resultado de transacciones equivale a algún orden serial, sin exigir orden real externo
consistencia externa	serializabilidad más respeto del orden observado por los clientes

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

Quórums, versiones y Dynamo

Mayoría en consenso y quorum Dynamo-like no son sinónimos

NÚCLEO

	Consenso	**Dynamo-like**
pregunta	qué entrada autoritativa se elige	cuántas réplicas participan
unidad	posición/slot del log	lectura o escritura de una key
concurrencia	protocolo impide commits incompatibles	puede conservar versiones concurrentes
resultado	orden autoritativo	conjunto de versiones y política de merge
intersección	parte de reglas de seguridad	propiedad combinatoria de R, W, N

Por qué necesitamos intersección

NÚCLEO

Con $N = 5$, una escritura llega a $\{N1, N2\}$ y una lectura posterior consulta $\{N4, N5\}$. Los conjuntos son disjuntos: la lectura puede no encontrar ninguna copia actualizada.

Queremos que todo conjunto ideal de lectura comparta al menos un nodo con todo conjunto ideal de escritura.

N, R y W: construir la intersección

ACTIVIDAD

- N: réplicas preferidas para la key.
- W: respuestas requeridas para aceptar la escritura.
- R: respuestas requeridas para completar la lectura.

$$R + W > N$$

Con $N = 5$, $W = 3$, $R = 3$, los conjuntos ideales deben intersectarse.

La intersección no crea automáticamente linearizabilidad, líder, orden total ni resolución semántica.

Contraejemplo 1: escrituras concurrentes

NÚCLEO

Dos clientes escriben versiones distintas sin conocer la otra. El nodo de intersección puede conservar ambas ramas.

Por qué contradice la conclusión ingenua

$R + W > N$ permite descubrir que existen versiones distintas, pero no decide cuál es posterior si fueron concurrentes.

Contraejemplo 2: sloppy quorum

NÚCLEO

Durante una falla, la escritura se almacena temporalmente en nodos fuera del conjunto preferido. Antes del hinted handoff, una lectura sobre las réplicas preferidas puede no cruzarse con el conjunto efectivo de escritura.

Lección

La fórmula usa conjuntos ideales; la implementación real debe considerar dónde quedaron efectivamente las copias.

Contraejemplo 3: last-write-wins y relojes desalineados

NÚCLEO

Una actualización causalmente posterior recibe un timestamp físico menor por desalineación del reloj y puede ser descartada como «más vieja».

Lección

Intersección y timestamps físicos no sustituyen un modelo correcto de causalidad.

Vector clocks reaparecen como genealogía del carrito

NÚCLEO

Partimos de:

$$v_0 = [0, 0]$$

Durante la partición:

$$\text{BA agrega libro} \rightarrow [1, 0]$$

$$\text{Madrid agrega auriculares} \rightarrow [0, 1]$$

Ninguna versión domina a la otra: son ramas concurrentes.

Carrito fusionable; entrada VIP no fusionable

NÚCLEO

Carrito

Puede conservar libro y auriculares y producir una versión descendiente que incluya ambas intenciones.

VIP-01

No existe un merge válido «Sofía + Tomás». El invariante exige una única decisión autoritativa.

Dynamo: localizar, escribir, conservar y reparar

1. aplicar hash a la key y construir una preference list;
2. escribir en réplicas hasta reunir W respuestas;
3. conservar versiones concurrentes en vez de perderlas;
4. responder según la política de disponibilidad;
5. reparar la divergencia en segundo plano o durante lecturas.

Sloppy quorum, hint y hinted handoff

NÚCLEO

- **Sloppy quorum**: aceptar temporalmente la copia en un nodo alternativo.
- **Hint**: metadato que recuerda cuál era el propietario preferido.
- **Hinted handoff**: devolver la copia al propietario cuando se recupera.

La disponibilidad desplaza costo hacia almacenamiento temporal, red, I/O y reparación posterior.

Read repair, anti-entropy y tasa de divergencia

NÚCLEO

- **Read repair**: corrige las réplicas consultadas al detectar versiones atrasadas.
- **Anti-entropy**: compara rangos en background para reparar datos fríos.
- **Tasa de divergencia**: diferencias nuevas creadas por unidad de tiempo.
- **Capacidad de reparación**: diferencias corregidas por unidad de tiempo.

$$D_{nuevo} > R_{resuelto} \Rightarrow \text{la deuda crece}$$

Por qué Dynamo aceptó esa complejidad

CASO REAL

El Dynamo original incorporó consistent hashing, preference lists, quórums, versionado, sloppy quorum, hinted handoff, read repair y anti-entropy para sostener la prioridad *always-on* de algunos servicios de Amazon.

No son sinónimos de «complejidad»; cada mecanismo paga una parte concreta de la disponibilidad durante fallas.

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

Spanner, TrueTime y orden externo

Sistemas Distribuidos

Por qué aparece Spanner en la misma historia

NÚCLEO

La compra de VIP-01 puede involucrar:

- grupo de entradas;
- grupo de pagos;
- grupo de derechos del usuario.

La operación debe ser atómica entre grupos y, si Madrid inicia una acción después de observar la confirmación de Buenos Aires, el orden externo debe respetarse.

Spanner: consenso localizado y transacción multi-grupo

NÚCLEO

Cada grupo de datos mantiene replicación y consenso propios. Una transacción que toca varios grupos coordina solo los grupos necesarios y asigna un timestamp de commit común.

El objetivo no es un único líder global para toda la base, sino coordinación localizada con una garantía transaccional global.

TrueTime: intervalo de tiempo con incertidumbre explícita

Spanner sincroniza servidores con referencias como GPS y relojes atómicos mediante time masters. Entre sincronizaciones existe deriva.

$$TT.now() = [earliest, latest]$$

Conceptualmente:

$$earliest = \widehat{t} - \epsilon, \qquad latest = \widehat{t} + \epsilon$$

TrueTime no afirma conocer un instante exacto: expone una cota que incluye error de sincronización y deriva.

Quién asigna el timestamp de commit

El coordinador de la transacción elige s respetando:

- timestamps de preparación de los grupos;
- la cota superior de TrueTime;
- dependencias y restricciones de la transacción.

Ejemplo:

$$TT.now() = [4, 16] \text{ ms}, \qquad s = 16 \text{ ms}$$

Commit wait: esperar hasta demostrar que s quedó en el pasado

NÚCLEO

El sistema espera hasta observar:

$$TT.now().earliest > s$$

Por ejemplo:

$$TT.now() = [17, 29] \text{ ms} \Rightarrow 17 > 16$$

La espera convierte incertidumbre temporal acotada en una garantía de orden externo.

Por qué se llama consistencia externa

Si un cliente recibe el commit de T_1 y después inicia T_2, toda serialización válida debe respetar:

$$T_1 < T_2$$

Se llama externa porque respeta el orden observado por clientes fuera del motor, no solo algún orden serial internamente válido.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Integración y cierre

Sistemas Distribuidos

Cada mecanismo responde una pregunta distinta

NÚCLEO

Mecanismo	Pregunta
Lamport/vector	¿qué eventos están relacionados o son concurrentes?
Raft/Paxos	¿qué comando se convierte en decisión autoritativa?
WAL local	¿qué recuerda cada nodo después de un crash?
Kafka	¿qué hechos pueden leer y reprocesar otros componentes?
CAP/PACELC	¿qué error y qué costo aceptamos bajo red imperfecta?
Dynamo	¿cómo seguir respondiendo y reparar divergencia?
Spanner	¿cómo sostener transacciones globales y orden externo?

Actividad integradora: cuatro operaciones, cuatro decisiones

ACTIVIDAD

Para cada operación declare invariante, mecanismo, comportamiento bajo partición, costo normal y prueba de falla:

1. última entrada VIP;
2. carrito compartido;
3. líder del control plane;
4. ranking global que entrega premios.

Exit ticket

ACTIVIDAD

Responda en una frase cada punto:

1. ¿Por qué $L(a) < L(b)$ no prueba causalidad?
2. ¿Cómo detecta un vector que dos eventos son concurrentes?
3. ¿Cuándo queda committed la entrada 87 en Raft?
4. ¿Qué diferencia hay entre log de consenso, WAL y Kafka?
5. ¿Por qué $R + W > N$ no implica linearizabilidad?
6. ¿Qué costo paga Spanner para obtener orden externo?

Bibliografía guiada

RESPALDO

- Kleppmann, *Designing Data-Intensive Applications*, caps. 5, 8 y 9.
- Lamport, *Time, Clocks, and the Ordering of Events*.
- Ongaro y Ousterhout, *In Search of an Understandable Consensus Algorithm*.
- Lamport, *Paxos Made Simple*.
- DeCandia et al., *Dynamo*.
- Corbett et al., *Spanner*.